Neurolib 2: A JAX-Based GPU-Accelerated Framework for Whole-Brain Neural Mass Modeling

Mohammad Shoorabisani
Master's Student in Computational Neuroscience
Bernstein Center for Computational Neuroscience (BCCN)

February 2026

1 Introduction and Background

Neural mass models describe the collective dynamics of large populations of neurons (often grouped by brain region) as low-dimensional systems. At the mesoscopic scale – intermediate between microscopic single-neuron and macroscopic whole-brain levels – these models capture mean activity (e.g. firing rates or average membrane potentials) of local populations . A classic example is the Wilson–Cowan model, which couples excitatory (E) and inhibitory (I) populations in each region using sigmoidal activation functions . This E/I architecture embodies the excitation–inhibition balance thought critical for stable cortical function . Balanced E/I drive maintains neurons in an appropriate operating range, so that shifts in excitation or inhibition produce changes in excitability . In whole-brain modeling, each brain region (or "mass") is typically treated as a homogeneous node with interacting subpopulations (E and I) that generate dynamic activity.

Modern brain modeling often connects these neural mass units into a network whose structure is given by the brain's connectome. The connectome is the comprehensive map of neural connections in the brain, representing the "wiring diagram" among regions . In practice, large-scale connectomes are derived from diffusion-weighted MRI (diffusion tensor imaging, or DTI) and tractography, which noninvasively trace white-matter fiber pathways in the living human brain . The output is a structural connectivity matrix, whose entries encode the strength (e.g. fiber count or probability) of connections between region pairs (and often associated fiber lengths for conduction delays) . This structural connectivity constrains how activity flows between regions, shaping network dynamics.

Neural mass models often aim to link these internal dynamics to observable signals. One common readout is the blood-oxygen-level-dependent (BOLD) signal measured by functional MRI (fMRI). The BOLD signal reflects local changes in blood oxygenation caused by neural activity and hemodynamic responses . In whole-brain models, neural activity (e.g. firing rates) can be transformed via a hemodynamic model into a synthetic BOLD time series, enabling direct comparison with fMRI data.

In this work we begin developing Neurolib 2, a GPU-accelerated, JAX-based successor to Neurolib 1, aimed at faster and more maintainable whole-brain simulations. We concentrate on the new code in base.py and wilsonCowanModel.py, which define the generic neural-mass model structure and a specific Wilson– Cowan network model, respectively. Our goals are to utilize modern tools (JAX for auto-differentiation and GPU support; Diffrax for differential-equation solvers) to speed up simulations, while preserving the established modeling concepts. Preliminary testing shows that even this early version of Neurolib 2 can reproduce the deterministic

dynamics of Neurolib 1's Wilson–Cowan model with high accuracy (within numerical tolerance), suggesting the approach is valid and efficient (see Testing and Comparison below).

2 The Brain as a Network: DTI and Fiber Tractography

At the macroscale, the brain can be viewed as a complex network of regions linked by white-matter fibers. Modern neuroimaging uses diffusion-weighted MRI (DW-MRI) to infer these connections. In diffusion tensor imaging (DTI), the diffusion of water molecules is measured along many directions; anisotropic diffusion in each voxel indicates underlying fiber orientation. Sophisticated reconstruction algorithms then perform fiber tractography, essentially chaining together preferred diffusion directions voxel by voxel to trace out putative neural pathways. DTI and tractography thus provide a noninvasive window into the brain's structural connectivity .

The result of tractography is often distilled into a structural connectivity (SC) matrix. One first defines a set of brain regions or Regions of Interest (ROIs), e.g. anatomical areas. Then for each ordered pair of ROIs (i, j), tractography yields a count or probability of streamlines (axon-like trajectories) connecting i to j. These values form the elements of a weighted adjacency matrix. In practice, the SC matrix entry C_{ij} may represent the fraction or number of fibers from region j that reach region i . Because tractography can also estimate the path length of each fiber, one often computes an associated fiber length matrix L_{ij} giving the average conduction distance between regions. These lengths, together with an assumed signal propagation speed, define an interregional delay matrix $D_{ij} = L_{ij}/v$, capturing the finite transmission time of neural signals in the network.

Structural connectivity matrices are sometimes called the "highways" of the brain : they constrain how activity in one region can influence distant regions. Many modeling studies have shown that the SC matrix strongly shapes emergent dynamics, including the observed patterns of functional connectivity (statistical correlations) in fMRI and EEG . In our Neurolib models, the connectivity matrix (fiber counts or weights) and length matrix (fiber lengths) are loaded as key model parameters. For example, in the BaseModel we store $fiber\_count\_matrix$ and $fiber\_length\_matrix$ and construct $connectivity\_matrix$ by zeroing self-connections (diagonals), as in other network models.

3 Connectivity Matrix: Strengths, Lengths, and Delays

The connectivity matrix in whole-brain modeling encodes the coupling between regions. Typical entries include a measure of strength (e.g. number or probability of fibers) and sometimes distance or length. These two pieces of information serve different roles:

- **Strength (weights):** Larger weights correspond to stronger inter-regional coupling. In practice, weights can be proportional to the fiber count or to the fraction of streamlines connecting the two regions. The absolute scale of the weights may be normalized or scaled to set an overall coupling level. Stronger coupling tends to synchronize activity across regions, while weaker coupling preserves more independent dynamics.
- **Length (delays):** The average length of fibers between regions is used to compute signal conduction delays. Neural signals travel at finite speed (e.g. 1–20 m/s). By dividing the fiber length by a signal propagation speed, one obtains a delay time for that connection. In code, we typically compute a delay matrix (same shape as the connectivity matrix) that enters the dynamical equations: inputs from region j affect region i with a delay of

D_{ij} seconds. Delays can qualitatively alter dynamics, e.g. by creating oscillatory modes or cluster synchronization.

In the BaseModel implementation, the input arguments include a $fiber\_count\_matrix$ and a $fiber\_length\_matrix$. From $fiber\_count\_matrix$ we derive the adjacency (connectivity) matrix by setting diagonal elements to zero (no self-coupling). Then a delay matrix is computed via a helper function (e.g. $compute\_delay\_matrix(fiber\_length\_matrix$, speed)) that yields $D_{ij} = L_{ij}/v$. For computational efficiency, we flatten and take the unique delay values and use diffrax's built-in Delay mechanism, which handles possibly many distinct delay lengths. (The code precomputes unique delays and an index mapping to speed up the solver.)

Because the SC matrix typically comes from real data (DTI of a subject or population), it can be quite large (e.g. hundreds of ROIs). Our BaseModel supports an arbitrary square connectivity matrix. We also perform some sanity checks (ensuring the matrix is square) and store $number\_of\_regions$ to manage array dimensions. By keeping fiber counts and lengths as JAX arrays (jax.numpy), the model can leverage JAX's JIT compilation and GPU execution when performing the simulation.

4 Neurolib 2 Base Model (`base.py`)

The BaseModel class (in base.py) defines the common structure and simulation method for any neural- mass model in Neurolib 2. It is built as an Equinox Module – a PyTorch-like class whose fields are PyTrees of JAX arrays. Key attributes of BaseModel include:

- state : a JAX array holding the current state variables (e.g. excitatory and inhibitory activities for all regions).
- dt : the time step (seconds) for integration.
- $fiber\_count\_matrix$: the raw connectivity weights.
- $fiber\_length\_matrix$: the inter-regional fiber lengths.
- $connectivity\_matrix$: derived from $fiber\_count\_matrix$ with diagonal zeroed.
- $delay\_matrix$, $unique\_delays$, $delay\_index\_matrix$: computed from lengths and propagation speed.
- $signal\_propagation\_speed$: scalar, used to compute delays.
- key : a JAX random key (for stochastic simulations, seeded).

Upon initialization, the BaseModel asserts that the connectivity and length matrices are square, stores the number of regions, and precomputes the delay matrix. Notably, delays can vary across connections; the code flattens the delay matrix, finds unique values, and creates an index matrix mapping each (i,j) to an index in the unique-delay list. This allows diffrax's Delay handling to efficiently reuse repeated delays.

The core dynamical structure is defined in abstract methods:

- dynamics(self, t, y, args, *, history) : the time derivatives for the state variables, typically supplied by a subclass. In BaseModel this is unimplemented (raises NotImplementedError).

- *get_term*(self, ts) : returns a diffrax term (ODE or SDE) for integration. The default implementation simply returns an ODETerm(self.dynamics) (no noise). Subclasses may override this to include noise.
- *history_fn*(self, t) : a function providing past states for t<0 if needed (for delays). Default is unimplemented; in models with delays, a history (often zero or steady-state) must be provided.

The simulate method in BaseModel orchestrates time integration. Given a total duration , it constructs a time array ts = arange(0, duration, dt) and calls diffrax.diffeqsolve . Key details:

- It uses the StratonovichMilstein solver from diffrax, which can solve SDEs in the Stratonovich sense. (Even if no noise is present, this choice is acceptable for ODEs.)
- It passes term = self.*get_term*(ts) to the solver. If noise is included via MultiTerm (see Wilson–Cowan below), it will handle it appropriately.
- It sets up diffrax.Delays with lambdas that return each unique delay value. delays=[lambda ...] ensures the solver accounts for all needed delay times, with an initial discontinuity at 0.
- The initial history function y0 is given by self.*history_fn*(t) ; the code assumes *history_fn* returns zero unless overridden.
- It collects the solution, then returns (sol.ts, sol.ys) as NumPy arrays for further analysis.

Importantly, because everything is written with JAX (and diffrax under the hood uses JAX) the simulation can be JIT-compiled and run on a GPU if available. Using JAX arrays for state and parameters means the model is fully differentiable. In fact, BaseModel provides a method *derivative_model*(self, x, *with_respect_to*) that can compute gradients of the simulation output with respect to model parameters. This uses Equinox's partition functionality to split the model into "differentiable" and "static" parts and then applies jax.grad to the sum of outputs. This allows parameter optimization or sensitivity analysis, a major advantage of automatic differentiation.

In summary, the base class encapsulates the network and integration machinery in a clean, modular way. It mirrors the architecture of Neurolib 1 but leverages JAX/Equinox for compatibility and performance. Using diffrax's advanced solvers and delay management simplifies the code: the complex loop over time with manual RK or Euler schemes is replaced by a single diffeqsolve call. Moreover, diffrax supports adaptive timestepping and SDEs natively, which enhances reliability and flexibility.

5 Wilson–Cowan Model Implementation (`wilsonCowanModel.py`)

The `WilsonCowan` class implements a classic two-population neural mass at each brain region, inheriting from `BaseModel`. It defines the excitatory–inhibitory (E–I) dynamics and parameters specific to the Wilson–Cowan equations. Some features of this implementation include:

- **Parameters (as JAX arrays):** The model holds many parameters per region, such as time constants (τ_e, τ_i), self- and cross-coupling weights ($w_{ee}, w_{ei}, w_{ie}, w_{ii}$), external inputs ($P_e, P_i$) and baseline external currents, and sigmoid parameters ($a_e, a_i, \theta_e, \theta_i$). By converting all parameters to JAX arrays in `__init__`, we ensure they are part of the model's PyTree and can be differentiated.

- **Logistic Activation Functions:** The code defines

$$\text{logistic}(x, a, \theta) = \frac{1}{1 + \exp(-a(x - \theta))},$$

and then wraps it for excitatory (S_e) and inhibitory (S_i) populations. These functions map total input into a bounded firing rate between 0 and 1, as typical in Wilson–Cowan models.

- **State Representation:** For N regions, the state vector $y \in \mathbb{R}^{2N}$ is split into excitatory and inhibitory parts:

```
exc = y[:N]      # Excitatory activity
inh = y[N:2N]    # Inhibitory activity
```

- **Delayed Coupling:** The excitatory equation includes delayed inputs from other regions. In `dynamics()`, the code accesses a history array of past states indexed by a precomputed `delay_index_matrix`. The inter-regional input is computed as

$$\text{exc\_interareal\_input} = \sum_j C_{ij}\, E_j(t - D_{ij}),$$

where C is the connectivity matrix and D_{ij} are conduction delays. No inter-regional inhibitory coupling is included.

- **Dynamics Equations:** For each region i, the deterministic Wilson–Cowan dynamics are

$$\tau_e \frac{dE_i}{dt} = -E_i + (1 - E_i)\, S_e(w_{ee}E_i - w_{ei}I_i + P_e + \text{baseline} + \text{input from other regions}), \tag{1}$$

$$\tau_i \frac{dI_i}{dt} = -I_i + (1 - I_i)\, S_i(w_{ie}E_i - w_{ii}I_i + P_i + \text{baseline}). \tag{2}$$

The code constructs the deterministic right-hand sides and returns the stacked derivative (dE/dt, dI/dt).

- **Noise Inclusion:** `WilsonCowan` overrides `get_term()` to include Ornstein–Uhlenbeck (OU) noise. The full state therefore contains both neural populations (E, I) and auxiliary noise variables.

- **History Function:** The `history_fn` method returns zeros for all $2N$ state variables, meaning the past state for $t < 0$ is assumed to be zero.

- **Simulation:** Calling `simulate()` uses the overridden `get_term()` to build a `diffrax.MultiTerm` consisting of:

  - an `ODETerm` for deterministic drift,
  - a `ControlTerm` for stochastic noise.

  The system is then integrated by Diffrax.

Overall, this class mirrors the standard Wilson–Cowan equations in a vectorized, JAX-compatible form across N regions. All computations use `jax.numpy`, enabling compilation, automatic differentiation, and parallel execution on accelerator hardware.

6 Noise and Ornstein–Uhlenbeck Processes

Real neuronal networks are inherently noisy. Background synaptic activity, channel noise, and other stochastic fluctuations affect population dynamics. To capture some of this variability, many neural mass models include random perturbations. In our Wilson–Cowan implementation we add Ornstein–Uhlenbeck (OU) noise to each population. An OU process is a mean-reverting stochastic process satisfying:

$$dX(t) = -\lambda(X(t) - \mu)\,dt + \sigma\,dW(t), \tag{3}$$

where λ is a rate constant pulling X toward its mean μ, and σ scales the white- noise input $dW(t)$. The OU process produces colored noise with exponential autocorrelation time $1/\lambda$, making it more biologically plausible than uncorrelated white noise.

In cortical models, OU noise is often used to approximate the effect of fluctuating synaptic inputs. For example, Fellous et al. (2003) simulated in vivo–like synaptic bombardment using OU processes for excitatory and inhibitory conductances. Similarly, we include an OU term for each of our excitatory and inhibitory populations. These noise terms allow the model to explore variability around deterministic trajectories.

Implementation in code: In `WilsonCowan.get_term()`, we construct a combined drift and diffusion term. We stack the neural state and the OU noise state into a single vector. The code defines:

- OU drift: a function `drift(t,y)` that pulls the OU variables y toward their means (`mean_exc_ou`, `mean_inh_ou`) with rate `tau_ou`. (Here the code uses a negative sign convention: $-$`tau_ou` $*$ ($y -$ `ou_means`).)
- Total drift: a function `stacked_drift` that returns the sum of the neural dynamics $(dE/dt, dI/dt)$ plus the OU contributions. The OU variables themselves evolve according to `drift()`, while the neural variables get an additional term from the instantaneous OU state (the noise can be interpreted as an additive current).
- Noise diffusion: An instance of `lineax_utils.OULinearOperator` is created with the OU time constant `tau_ou` and noise amplitude `sigma_ou`. This linear operator encodes the covariance structure of the OU process for multiple dimensions (here 2$*$`number_of_regions`).
- Brownian motion: We define a `diffrax.BrownianMotion` object with the same shape as the OU state, which generates the Wiener increments over time.

The solver term is then
`diffrax.MultiTerm(diffrax.ODETerm(stacked_drift), diffrax.ControlTerm(diffusion, brownian_motion))`.

This tells diffrax to integrate the drift (`stacked_drift`) and add stochastic forcing via `diffusion` $* dW$. In practice, this implements the OU SDE.

The result is that each excitatory and inhibitory population receives temporally correlated random input. Having two OU processes (one for E and one for I) per region adds rich dynamics; the mean-reverting nature of OU ensures the noise does not wander unbounded, and $\sigma_{\texttt{ou}}$ controls the variance. The advantage of including OU rather than white noise is that it has a finite correlation time, which can better mimic synaptic filtering. Also, because diffrax handles SDEs in Stratonovich form, the numerical integration remains stable and accurate even with noise.

We have implemented two OU noise terms (one for each population). It is possible the goal referred to having one noise source per population (hence two, not three). If a third noise were needed (e.g. global input noise), the framework could easily add another OU process.

7 Testing and Comparison with Neurolib 1

To verify the correctness of our implementation, we performed a deterministic comparison against the original Neurolib (version 1) Wilson–Cowan model. Specifically, we ran both models in a simple scenario: a single isolated node ($N = 1$) with no coupling to other regions and no noise. Under these conditions, the network equations reduce to an uncoupled two-population Wilson–Cowan system.

The test (`wc_deterministic.test.py`) proceeds as follows:

1. Set parameters (time constants, weights, biases, etc.) to identical values in both models.
2. Run Neurolib 1: Create a Neurolib WCModel with zero coupling and zero noise, simulate it with a standard solver (Neurolib likely uses a fixed-step Euler method).
3. Run Neurolib 2 (JAX/Diffrax): Instantiate our WilsonCowan model with the same parameters and call its `simulate()`.
4. Compare outputs: Align the time series (using interpolation if needed) and compute the absolute differences in E and I time courses.
5. Check tolerance: Verify that differences are below a strict tolerance (e.g. relative tolerance 10^{-3}, absolute 10^{-4}).

8 Conclusion

We have presented the foundations of Neurolib 2, focusing on the BaseModel and Wilson–Cowan model classes. By harnessing JAX, Equinox, and Diffrax, we have reimplemented neural-mass models with GPU- accelerated solvers and built-in differentiation. Our results show that even at this early stage, the new code reproduces established Wilson–Cowan dynamics (in the no-noise limit) with high fidelity.

Key advantages of this approach include:

- **Speed and parallelism:** JAX allows automatic JIT compilation and parallel execution on GPUs or TPUs. The vectorized implementation can handle many regions simultaneously without Python loops.
- **Reliability and maintainability:** Using diffrax's integrators and JAX's functional style reduces custom solver code, lowering the risk of bugs. The code is organized with clear class structures (Equinox modules) and leverages mature libraries.
- **Automatic differentiation:** The ability to compute gradients of model outputs with respect to parameters (`derivative_model`) opens doors for systematic parameter fitting or sensitivity analysis, which was not straightforward in Neurolib 1.
- **Stochastic modeling:** Incorporating OU noise with diffrax allows easy experimentation with noisy dynamics, supporting more biologically realistic simulations.

Of course, this project is still in progress. Many features of Neurolib 1 (e.g. multiple built-in models, advanced coupling patterns, BOLD-forward models) remain to be ported or reimplemented. Nonetheless, the feasibility has been demonstrated. Future work will include extensive testing (including stochastic simulations), performance benchmarking against large-scale scenarios, and full integration of parameter sampling and fitting workflows.

In summary, Neurolib 2's new implementation shows that modern JAX-based tools can achieve the twin goals of speed and flexibility for whole-brain modeling. By grounding our methods in established neuroscience concepts (neural mass models, connectomes, hemodynamic signals) and using academically rigorous techniques, we have laid the groundwork for the next generation of large-scale brain simulations.

References

[1] Frontiers | On the Validity of Neural Mass Models.

[2] Wilson–Cowan model - Wikipedia.

[3] Neurophysiology and regulation of excitation/inhibition balance.

[4] Connectome - Wikipedia.

[5] Diffusion Tensor MR Imaging and Fiber Tractography.

[6] Human brain structural connectivity matrices.

[7] Human brain structural connectivity matrices–ready for modelling.

[8] Structural connectivity and network modeling studies.

[9] Blood-oxygenation-level–dependent imaging - Wikipedia.

[10] Equinox: neural networks in JAX via callable PyTrees.

[11] Diffrax documentation.

[12] Synaptic background noise and OU processes.